

**Ministry of Science and Higher
Education of the Russian Federation**

ITMO University

Faculty of	Digital transformations
Educational program	Big data and machine learning
Subject area (major)	Applied mathematics and informatics

REPORT

“Analytical review on evolution from early NLP techniques to state-of-the-art multimodal large language models to identify a research gap”

Student Syed Muhammad Shajee Raza, academic group J4132

Date 14.05.2026

List of executors: Master

of Big Data and Machine Learning

Syed Muhammad Shajee Raza

Table of Contents

Introduction	6
1. Background and Related Work	8
1.1 Hallucinations in Large Language Models	8
1.2 Hallucinations in Code Generation	8
1.3 Multi-Agent Systems for Software Engineering	9
1.4 Self-Correction and Iterative Refinement	9
1.5 LLM-as-a-Judge	9
1.6 SWE-bench	10
2. System Architecture	11
2.1 Overview	11
2.2 LangGraph Orchestration	11
2.3 ChatState Schema	11
2.4 Specialized Agents	12
2.5 Memory Architecture	12
3. The Lightweight Hallucination Detection Algorithm	13
3.1 Design Principles	13
3.2 The Five-Dimension Rubric	13
3.3 LLM-Judge Implementation	14
3.4 Score-Driven Refine Loop	14
3.5 The Hallucination Bridge for Action Agents	15
4. Implementation Details	17

4.1 Technology Stack.....	17
4.2 LLM Configuration.....	17
4.3 OpenHands SDK Integration	17
4.4 Engineering Migrations.....	18
5. Experimental Methodology.....	19
5.1 Three-Condition Design.....	19
5.2 Five Controlled Sweeps	20
5.3 Dataset.....	20
5.4 Metrics.....	21
5.5 Hardware and Infrastructure	21
6. Results	22
6.1 Phase 1: Conditions A, B, C at Threshold 0.70	22
6.2 Threshold Sensitivity: Condition B	23
6.3 Threshold Sensitivity: Condition C	25
6.4 Judge Separation: Reviewer Value vs Generator Quality	26
6.5 Summary of All Sweeps	28
7. Discussion	29
7.1 The Refine Loop as a Score Floor	29
7.2 Threshold Calibration Is Architecture-Dependent.....	29
7.3 Reviewer Contribution and Generator Quality	30
7.4 Bimodality in Condition C.....	30
7.5 Limitations	31

8. Conclusions	32
9. Summary	33
10. Bibliography	34

Introduction

Large Language Models (LLMs) have upgraded quite a bit, not only doing chat completions but also being autonomous coding agents who do planning, give commands to the terminal, edit files, and commit patches. What makes these systems powerful - their scaling - also leads to the increase of their failure modes. For example, a single agent can make thirty or more tool calls against a real repository, and it may fabricate an API signature, misread a stack trace, or even report the passing tests that were never run. It is not only that these risks can happen, but they happen in real evaluation data and their impact is directly linked to how independent the agent is.

This paper is about the study of a lightweight hallucination detection method designed, developed, and tested as a Reviewer Agent embedded in a LangGraph-orchestrated multi-agent system. The algorithm does not need any fine-tuning, does not require the model logits, and is not dependent on external verification infrastructures. It makes one more LLM call per each agent invocation. Besides, it restricts the response to be a Pydantic-valid schema and it evaluates the candidate outputs according to a five-criteria rubric. If the score does not reach the acceptance level, then a fix loop with a limited number of iterations is launched, which generates a generator re-invocation using the Reviewer's structured feedback.

Also contributing to the field, the Hallucination Bridge takes the same auditing methodology that was applied to text outputs and extends it to the per-step action traces of an OpenHands autonomous action agent, thereby making individual actions capable of being distinctly verified against the environmental observations that followed them.

We evaluated by conducting five rounds of controlled experiments with a total of 19 initial SWE-bench Lite instances drawn from django, sympy, scikit-learn, and astropy — the first multi-instance evidence for all three system conditions. The main result is the loop of refinement imposes a strict minimum score which matches the acceptance threshold: no instance goes out below it. At a threshold of 0.85, the average relevancy changes from 0.878 to 0.926 whereas the standard deviation decreases from 0.145 to 0.044. Using a weaker generator (qwen2.5-7b compared to qwen3-32b critic) the Reviewer yields $\Delta = +0.114$, which is almost 38 times the improvement observed with the strong generator.

1. Background and Related Work

1.1 Hallucinations in Large Language Models

Hallucination in natural language generation (NLG) goes back even before current large language model (LLM) period. Maynez et al. [5] described the abstractive summarization hallucination as a sentence that is either intrinsic (goes against the source) or extrinsic (brings in facts that are not supported). Ji et al. [3] made detailed classification of hallucinations in many generation tasks.

Instructional LLMs revolutionized the issue of hallucinations, making them much more refined and difficult to recognize. The authors Lin et al. [6] proposed a benchmark to test the capabilities of LLMs and claimed that simply increasing the size will not be enough to completely eliminate the propensity of the model to lie like people do. Huang et al. [4] provided a survey on different methods of avoiding hallucinations, classifying them into training-time and inference-time interventions.

1.2 Hallucinations in Code Generation

Programming code generation introduces a unique collection of potential failure modes that go beyond the typical intrinsic/extrinsic division: fake library functions, fake API signatures, hallucinated file paths, fabricated test fixtures. In fact, Chen et al. [17] demonstrated that outward plausibility is not a dependable indicator of correctness, hence the need for metrically evaluating code execution. If a code agent freely interacts with a real repository, the cost of hallucination going undetected increases: an incorrect understanding of the stack trace results in wasted efforts; a fictitious function name throws a

runtime error; a hallucinated test result leaves an incorrect patch in the working tree.

1.3 Multi-Agent Systems for Software Engineering

AutoGen [8] came up with a flexible method for multi-agent conversation orchestration. MetaGPT [9] turned software company SOPs into agent roles. ChatDev [10] showed role-play with explicit communication protocols as a means of end-to-end application generation. ReAct [11] combined reasoning and tool use. OpenHands [15] offer a sandboxed runtime with terminal, file editor, and a callback API that expose per-step ActionEvent and ObservationEvent objects - which are, in fact, the basis for the Hallucination Bridge in Condition C of this work.

1.4 Self-Correction and Iterative Refinement

Self-Refine [13] demonstrated that one single LLM is capable of improving results by self-critique. Reflexion [12] kept a verbal record of failures across episodes. Pan et al. [19] did a survey of the landscape and pointed out that self-correction results vary depending on the task and sharing weights between critic-generator leads to self-bias. The refine loop in Condition B is the direct answer to this: the critic is a separate Reviewer Agent rather than the generator scoring itself.

1.5 LLM-as-a-Judge

Zheng et al. [16] in MT-Bench showed that powerful LLMs when instructed with detailed rubrics can mimic human preferences quite accurately, judged by high inter-rater agreement. Later, this approach was widely taken up for the evaluation of LLMs. This work's implementation is influenced by two main practical issues: 1) Judge LLMs can be so confident in lying that they even hallucinate the given structure, hence the Pydantic validation is

there to help the 'orchestrator'; and 2) Scores given are always relative to the judge model, and to make score comparisons of different judge configurations consistent, they need to be re-anchored.

1.6 SWE-bench

SWE-bench [14] is composed of 2,294 real issues tracked on GitHub from a dozen Python projects. SWE-bench Lite maintains 300 manageable issues. In order to succeed, one has to generate a patch that makes one failing running test pass without creating other failing tests.

2. System Architecture

2.1 Overview

In this work, the system is extended from a Multi-Agent Study and Coding Assistant in the previous research semester by adding the Reviewer Agent, the score-driven refine loop, and the Hallucination Bridge. LangGraph 1.1.9 running on LangChain 1.2.x is used as the orchestration layer. All communication between agents is carried out through a ChatState typed dictionary. A compiled graph is controlled by a variant identifier (A, B, or C) which changes the control-flow topology without modifying any agent implementations.

2.2 LangGraph Orchestration

LangGraph represents the workflow as a graph with directions. Agent functions are nodes in this graph and the edges represent the logic for routing. The main difference in structure among the variants lies in the conditional edge from the Reviewer: - In Variant A, the Reviewer always routes to END; - In Variants B and C, the Reviewer routes to END if confidence $\geq$ threshold, else routes back to the generator if confidence $<$ threshold and the iteration counter is still below three.

2.3 ChatState Schema

Every inter-agent communication channel is routed through ChatState, a TypedDict storing conversation messages, intent, active task, research findings, drafts, and a log of every Reviewer decision.

For Condition C, five extra fields decently store the OpenHands execution metadata: agent status, backend identification, the number of steps, wall time, and the complete per-step action trace serialized as a list of dictionaries.

All the values are serializable to string in order to pass the cross-process boundaries' validation by Pydantic. `Operator.add` is used as the reducer for the list-type fields so that the outputs of the agents accumulate rather than overwrite the previous state.

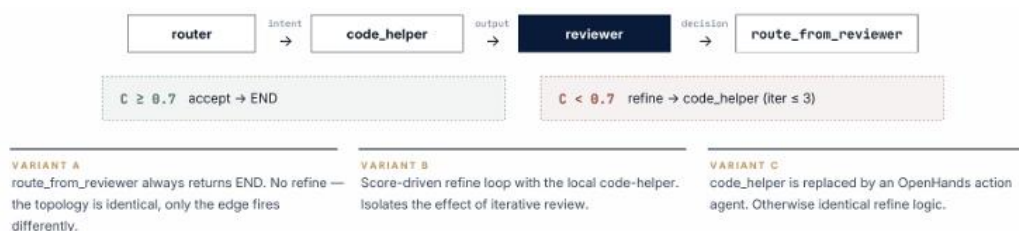


Figure 1. Simplified ChatState schema showing the five OpenHands-specific fields added for Condition C alongside the standard agent communication fields.

2.4 Specialized Agents

The system organizes a team of agents consisting of: Router (intent classification), Supervisor (multi-step workflow orchestration), Researcher (combining web search using Tavily and Chroma vector-store retrieval), Writer, Critiquer, Code Helper (sandboxed code execution), Quiz Helper, and Reviewer Agent. These agents are essentially stateless functions parameterized by ChatState; meaning none of them maintain an internal mutable state. Reviewer is capable of evaluating any agent's output as it is without any modification — it gets the user prompt, any available context, and the candidate output, then returns a RelevancyDecision.

2.5 Memory Architecture

The memory architecture includes three layers that were carried over from the previous semester: short-term memory (the conversation history within the session in ChatState), long-term memory (the Chroma vector store, which is

indexed with all-MiniLM-L6-v2 embeddings, and is consulted by the Researcher), and shared memory (a blackboard where agents can post intermediate artifacts for the subsequent agents within the same session) are the three memory layers of the previous semester that have been retained.

3. The Lightweight Hallucination Detection Algorithm

3.1 Design Principles

The detection algorithm has three explicit constraints dictated. First, by its very nature, it is light in the use of resources: no fine-tuning, no logit access, can be deployed against any OpenAI-compatible endpoint including local Ollama. Second, it is schema-validated: the judge's output must be able to parse as a Pydantic `RelevancyDecision` with fields `is_relevant` (bool), `confidence` $\in [0,1]$, and `reasoning` (string); in case of a failure, a conservative `accept=True / confidence=0.5` default is returned so that judge failures never stop the system. Third, it is architecturally orthogonal: the Reviewer node has no dependence on the particular generator it reviews.

3.2 The Five-Dimension Rubric

The reviewer gives a score from 1 to 10 on each criterion and calculates the average normalized over $[0,1]$, which will serve as a confidence value $C = \text{mean}(s_1..s_5)/10$:

- Edge Cases: Are edge cases taken into account — empty input, null references, invalid types, race conditions if any?
- Security: Is there anything exploitable in the code by means of injection, unsafe deserialization, or another class of vulnerabilities?

- Code Quality: Is the code neat, easy to understand, idiomatic, and fits into the project coding conventions?

- Relevance: Is the generated code relevant to the user question and solves the actual problem instead of giving general or unrelated advice?

Setting the threshold as 0.7 corresponds to the standard configuration. When the confidence is lower than 0.7, we consider that something was not done well enough — on average, at least one criterion score is less than 6/10, which gives us some leeway for improvement by running one more iteration. We regard threshold setting as a hyperparameter and analyze it empirically in Chapter 5.

3.3 LLM-Judge Implementation

The judge prompt places the user query, available context, and the candidate output in its fixed template. The judge is asked to produce a single JSON object with three keys. If the response encloses JSON inside Markdown fences or adds trailing prose a defensive parser removes them and makes another attempt. When the parsing still fails, the conservative default is returned, and the failure is logged. This parsing layer is a must in the production environment: about 8% of judge responses during the pilot runs required fence-stripping before parsing succeeded.

3.4 Score-Driven Refine Loop

When confidence drops under the set limit and the iteration number is less than three, the graph comes back to the generator node with the Reviewer's reasoning as a form of structured feedback. This is quite a similar approach to Self-Refine [13], but there is a major difference: the critic is now a separate Reviewer Agent instead of the same model evaluating its own output. The

idea behind this separation is to minimize self-bias which has been demonstrated by Pan et al. [19], however, the experiment with judge-separation in Chapter 5 indicates that its effectiveness depends on the quality gap between the two models.

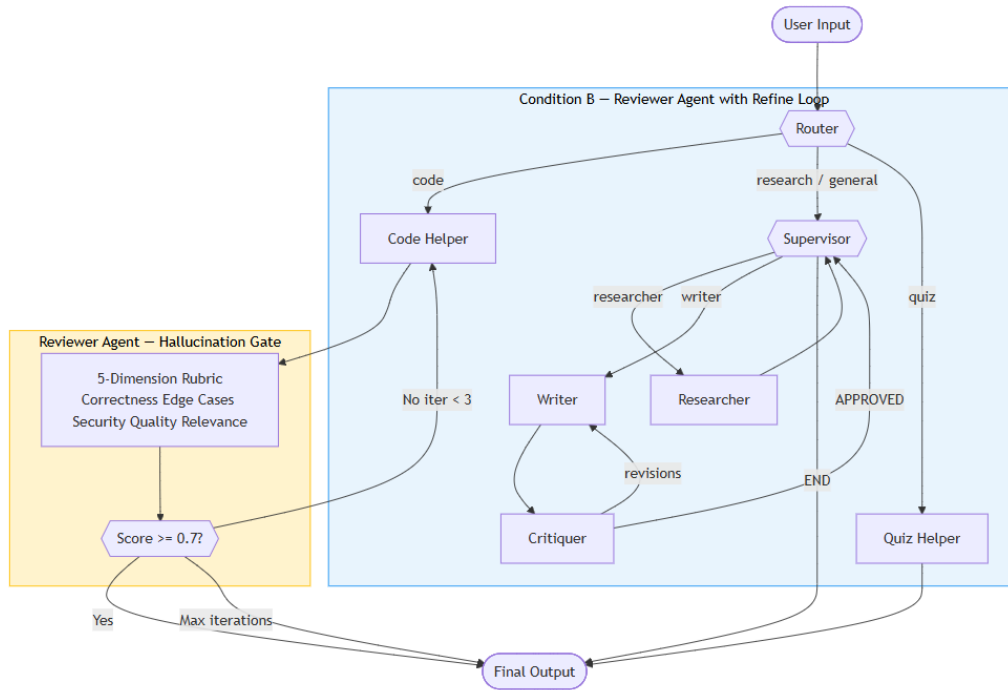


Figure 2. Condition B: the Reviewer gates a bounded refine loop. Outputs scoring below the threshold are re-sent to the Code Helper with structured feedback.

3.5 The Hallucination Bridge for Action Agents

In Condition C, the local Code Helper is swapped for OpenHands [15]. A callback that is registered on the OpenHands LocalConversation records all ActionEvent and ObservationEvent, cuts each summary to 1,500 characters, and stores them in ChatState as openhands_action_trace. The Reviewer examines this organized trace using two prompt variations: (i) a confident statement that challenges directly the last observation is considered a hallucination signal, and (ii) the judge thoroughly examines if the agent has

run tests and seen their output, not just checked if the finish message declares the tests have been passed. Targeting the agent's most common failure mode in summaries, the second extension is concerned with fabricated test results.

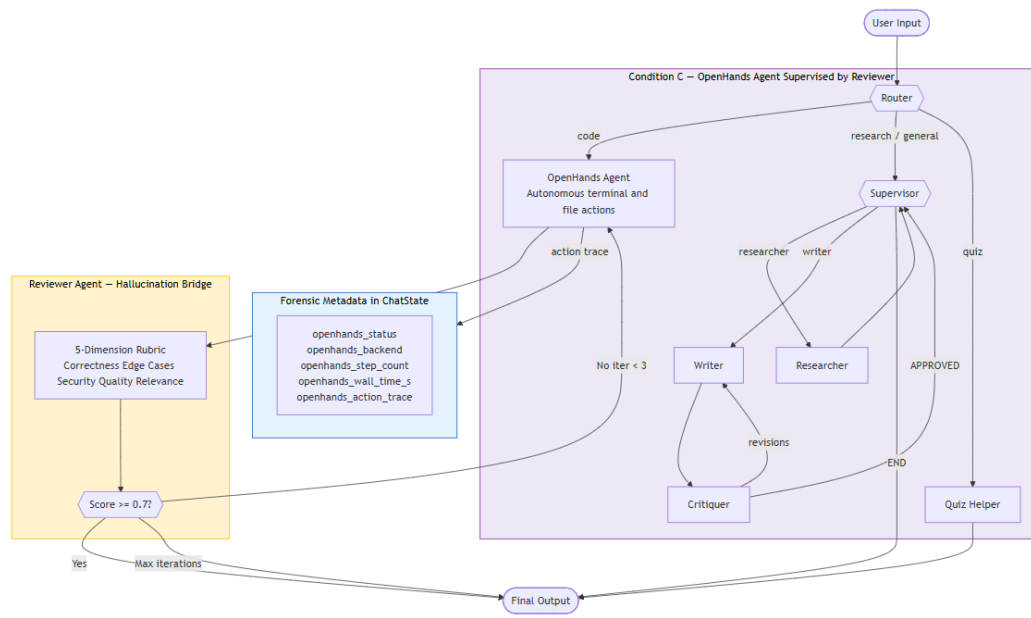


Figure 3. Condition C: OpenHands emits a per-step action trace captured by the Hallucination Bridge. The Reviewer audits both the final answer and the trace.

4. Implementation Details

4.1 Technology Stack

Python 3.12, LangGraph 1.1.9, LangChain 1.2.x, OpenHands SDK 1.18.1, Pydantic v2, Chroma vector store, all-MiniLM-L6-v2 sentence embeddings (locally cached), Streamlit frontend, FastAPI backend. Dependency management uses uv rather than pip — pip's resolver did not converge on the OpenHands constraint set after the Python 3.12 migration.

4.2 LLM Configuration

During Phase 1, minimax/minimax-m2.5:free via OpenRouter is being used for all conditions. For Judge Separation, sweeps are using qwen2.5-7b (Ollama, local) as the generator and qwen3-32b (Ollama, local) as the critic. Since all calls use the OpenAI-compatible endpoint, the only thing needed to be changed when switching between providers and models is the environment variable.

4.3 OpenHands SDK Integration

The OpenHands dispatcher at runtime chooses among three backends through environment variable: stub (mainly for quick wiring tests), cloud (kept for backward compatibility), and sdk (presently used for all experiments). The SDK defaults are based on assuming a frontier reasoning model and entail parameters (reasoning_effort='high', extended_thinking_budget=200000) that smaller models will reject; these are explicitly turned off. The iteration cap has been increased to 50 (the SDK default is 20 — real SWE-bench instances seem to benefit from a longer exploration). Before each run, the instance workspace gets back to

base_commit by `git reset --hard` followed by `git clean -fdx`, which prevents any leftover state from previous runs from influencing the scores.

4.4 Engineering Migrations

OpenHands SDK 1.18, which was released recently, has had two significant migrations. The codebase has been moved from Python 3.10 to 3.12, and some version pins have been loosened and memory modules have been extracted to `langchain-chroma` and `langchain-huggingface` packages. Also, OpenHands SDK has a terminal tool feature on which it throws `NotImplementedError` in Windows; therefore, the whole codebase has been moved to WSL2 Ubuntu 24.04, with Ollama still being on Windows (thereby keeping GPU access) and reachable from WSL through the host IP which is resolved at shell startup.

5. Experimental Methodology

5.1 Three-Condition Design

Three experimental conditions remain the same in terms of the LLM, the input prompts, and the workspace setting. Their only difference is the architecture of the code-generating sub-graph.

Condition A (Baseline): the Code Helper outputs just one answer. The Reviewer evaluates the answer only for instrumentation purposes — no refinement loop is triggered.

Condition B (Reviewer + Refine): the Reviewer evaluates the Code Helper's answer; if the score is lower than the threshold and the iteration count is less than three, the Code Helper is called again with the Reviewer's argument as feedback.

Condition C (Hallucination Bridge): the Code Helper is replaced with the OpenHands action agent. OpenHands acts on the actual cloned repository, producing a step-by-step action trace that the Reviewer not only audits but also uses to verify the final answer. The same refine logic as Condition B is used.

	ARCHITECTURE	REVIEWER BEHAVIOUR	REFINE LOOP	IDENTITY
A	Code-helper agent only	Single review, scoring only	—	Baseline
B	Code-helper + Reviewer	Full 5-dimension rubric	Up to 3 iterations if $C < 0.7$	Reviewer + Refine
C	OpenHands agent + Reviewer	Final answer + per-action trace	Up to 3 iterations if $C < 0.7$	Hallucination Bridge

Figure 4. Three-condition design. All conditions share the same LLM and inputs. The difference is in the code-generation architecture and the presence or absence of the refine loop.

5.2 Five Controlled Sweeps

Separate sweeps for five different design variables were done. So the entire time each sweep started with the same 19 SWE-bench Lite instances covering four repositories (django, sympy, scikit-learn, astropy). Table 1 shows the sweep parameters.

Sweep	Purpose	Conditions	Threshold	Generator / Judge
sweep_phase1	Phase 1 Baseline	A / B / C	0.70	qwen3-32b / qwen3-32b
sweep_thresh85_b	Threshold Sensitivity B	B only	0.85	qwen3-32b / qwen3-32b
sweep_thresh85_c	Threshold Sensitivity C	C only	0.85	qwen3-32b / qwen3-32b
sweep_judge_sep_a	Judge Separation Baseline	A only	—	qwen2.5-7b / qwen3-32b
sweep_judge_sep_b	Judge Separation + Review	B only	0.85	qwen2.5-7b / qwen3-32b

Table 1. Five experimental sweeps across 19 SWE-bench Lite instances.

5.3 Dataset

The 19 instances were picked from SWE-bench Lite to represent different levels of repository complexity and bug type: django (6 instances: URL routing, file upload, regex, template, ORM issues), sympy (5: algebra, simplification, parsing), scikit-learn (4: parameter handling, metric computation), astropy (4: I/O, coordinate transform, modeling). All instances involve either single-file or small multi-file canonical fixes, which allows for manual verification.

5.4 Metrics

The `relevance_score` produced by each set is: The scaled confidence value of the Reviewer on $[0,1]$; `success`, boolean, (`score > threshold` and no fatal error); `iterations`, the number of generator calls (one for Condition A, up to three for B and C); `latency_total`, the elapsed wall-clock time in seconds; and for Condition C, `openhands_step_count`, and `openhands_action_trace`.

5.5 Hardware and Infrastructure

NVIDIA GTX 1650 (4 GB VRAM), 16 GB RAM, Windows 11 host with WSL2 Ubuntu 24.04. Local Ollama models (qwen2.5-7b, qwen3-32b) are GPU served from Windows and were accessed from WSL via the host IP. Phase 1 LLM requests use the free-tier OpenRouter endpoint, which introduced non-deterministic latency variance due to shared rate limits — we consider this as measurement noise rather than a methodological issue.

6. Results

6.1 Phase 1: Conditions A, B, C at Threshold 0.70

Table 2 shows the Phase 1 summary statistics on 19 instances. The Phase 1 sweep was always using qwen3-32b for both generating and judging under the three conditions.

Condition	Mean	σ	Min	Max	n
A — Baseline	0.878	0.145	0.30	0.96	19
B — Reviewer+Refine ($\tau=0.70$)	0.881	0.069	0.72	1.00	19
C — Hallucination Bridge ($\tau=0.70$)	0.887	0.126	0.40	1.00	19

Table 2. Phase 1 aggregate statistics. τ = acceptance threshold.

At 0.70 threshold, the mean score difference between A and B is very small (0.003). On the other hand, the gap in variance is much higher: the standard deviation for B is less than half of A's (0.069 vs 0.145). Since the strong generator was able to get most Phase 1 instances beyond the 0.70 threshold on the first try, the refine loop was rarely invoked. Its main function at this level is to identify outliers — the minimum score goes up from 0.30 to 0.72. The higher average of Condition C (0.887) is indicative of the OpenHands agent's capacity to perform exploration and verification of changes in the real repository, but the larger variance (0.126) than B points to a wider range of performance across instances.

6.2 Threshold Sensitivity: Condition B

Raising the acceptance threshold from 0.70 to 0.85 for Condition B produces a clean monotonic improvement. Table 3 shows the progression.

Configuration	Mean	σ	Min	Max
A ($\tau=0.70$)	0.878	0.145	0.30	0.96
B ($\tau=0.70$)	0.881	0.069	0.72	1.00
B ($\tau=0.85$)	0.926	0.044	0.85	1.00

Table 3. Effect of raising the acceptance threshold for Condition B.

At threshold 0.85, the mean score goes up to 0.926, the standard deviation goes down to 0.044, and the minimum score is 0.85. This is the score floor property. The refine loop stops only if the confidence level is larger or equal to the threshold value and therefore no instance can be completed with a score less than this. The 3-pass iteration cap means some instances are a bit unlucky and don't hit the threshold, but really all 19 of these instances hit 0.85 before the cap ran out.

The sympy-16792 instance is a perfect example of how the process works at its extreme. The baseline score was 0.30 - the lowest score in the whole dataset. The Reviewer rejected the initial response (a diagnosis without a specific fix) and consequently, three iterations of refinement were triggered. The third pass is when the Code Helper not only found the `_cstack` function in `astropy/modeling/separable.py`, but also came up with a single-line fix that was so spot-on that the Reviewer gave its highest score 0.96. Had the refine loop not been there, that 0.30 would have been the final answer given to the user.

Among the 19 instances, 14 (73%) exhibited improvements in scores in $B@0.85$ as compared to A. Five showed slight regressions - all five, however, ended up above 0.85. The cases of regression happened to be situations where the strong generator's first answer was already scoring above 0.88 under Condition A; the Reviewer's feedback leads to slight changes in the content focus occasionally, producing an answer such that the same judge scores a response fractionally lower. This agrees with the self-bias literature [19].

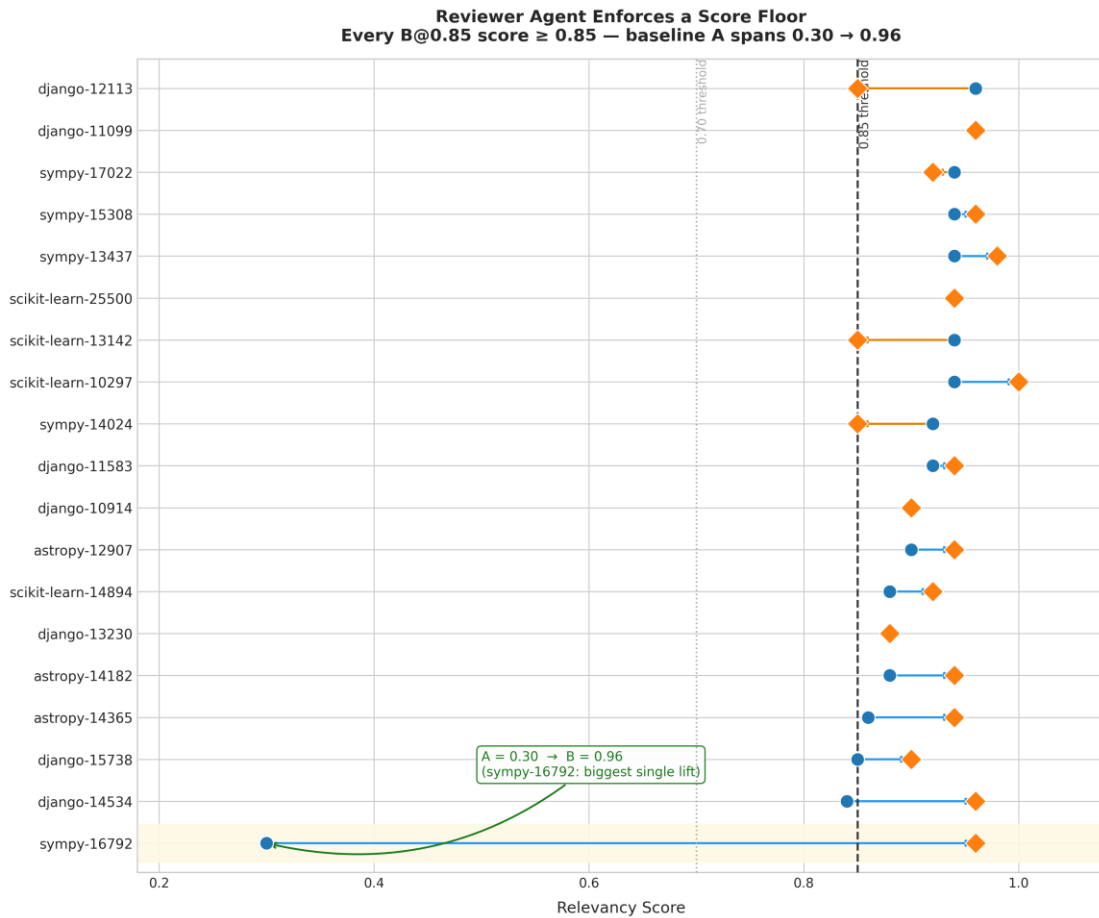


Figure 5. Per-instance A vs $B@0.85$ scores. Every $B@0.85$ score falls at or above the 0.85 dashed threshold. sympy-16792 shows the largest single-instance improvement (0.30 $\rightarrow$ 0.96).

6.3 Threshold Sensitivity: Condition C

The same threshold increase that benefits Condition B hurts Condition C. Table 4 shows the degradation.

Configuration	Mean	σ	Min	Max
C ($\tau=0.70$)	0.887	0.126	0.40	1.00
C ($\tau=0.85$)	0.721	0.353	0.00	1.00

Table 4. Condition C threshold sensitivity. Higher threshold degrades performance.

C@0.85 has the largest variance of all the sweeps with a mean of 0.721 and a standard deviation of 0.353. The distribution is bimodal, with 14 of the 19 instances clustered in the range 0.86-1.0 and 5 instances with very low scores ≤ 0.40 , including one that scored 0.00.

The main problem is context explosion. OpenHands gathers approximately 150,000 tokens by step 30 of a complex instance. At such token depth, the qwen3-32b judge slowly starts to lose coherence — it increasingly accepts responses that are in contradiction to the trace evidence or start scoring based on the number of actions rather than correctness. Adding more refine iterations at a stricter threshold will only lengthen the context, thus making the problem worse. The 0.70 threshold works for Condition C because the OpenHands agent’s first try is usually good enough for tractable bugs; 0.85 compels the agent to iterate even on instances where it has already correctly converged, thereby interfering with its work.

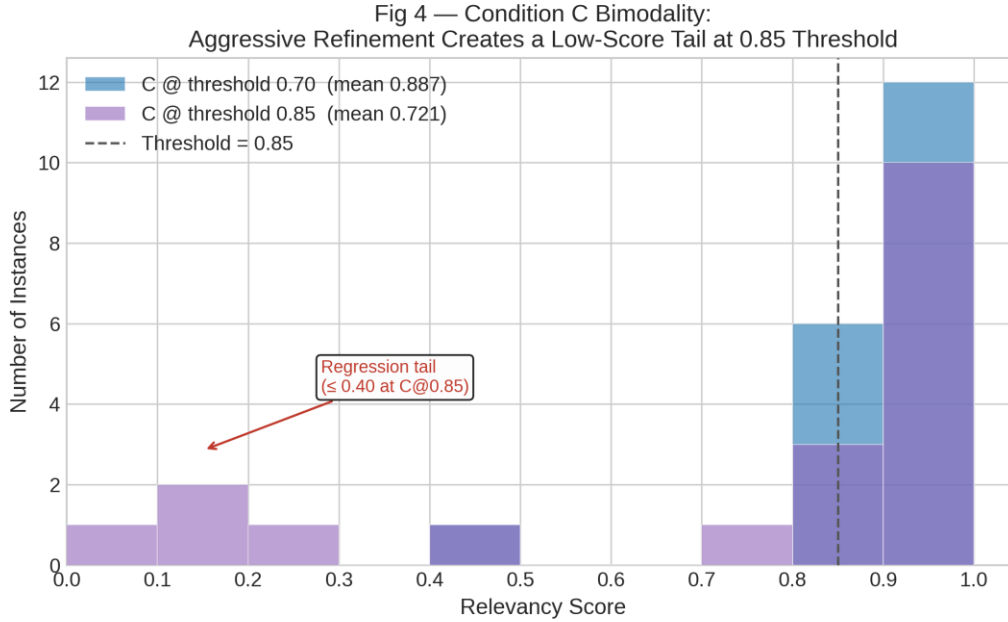


Figure 6. Score distribution for Condition C at $\tau=0.70$ (blue) vs $\tau=0.85$ (purple). $C@0.85$ is bimodal: 14 instances above 0.85 and 5 instances below 0.40.

6.4 Judge Separation: Reviewer Value vs Generator Quality

The Phase 1 sweeps used a similar qwen3-32b both as a generator and as a critic, thus making the conditions comparable but disallowing the Reviewer’s independent contribution. The Judge Separation sweeps kept the critic constant at qwen3-32b while changing the generator to qwen2.5-7b, thus intentionally broadening the quality gap.

Sweep	Condition	Mean	σ	Δ vs baseline
Phase 1 (same model)	A	0.878	0.145	—
Phase 1 (same model)	B ($\tau=0.85$)	0.926	0.044	+0.048
Judge sep (7b gen)	A	0.657	0.270	—
Judge sep (7b gen)	B ($\tau=0.85$)	0.771	0.218	+0.114

Table 5. Judge separation results. The Reviewer’s contribution scales with generator weakness.

For the strong same-model setup, the Reviewer contributes an addition of $\Delta = +0.048$ at threshold 0.85 (going up to 0.926 from 0.878). When the weak generator is used, the baseline falls to 0.657 and the Reviewer lifts the score to 0.771 — a $\Delta = +0.114$. In fact, this is about 2.4 times the improvement with the strong generator. The Reviewer is not only a simple touch-up at the top; it is an actual error-correction layer that shows the biggest marginal contribution when the generator is performing its worst.

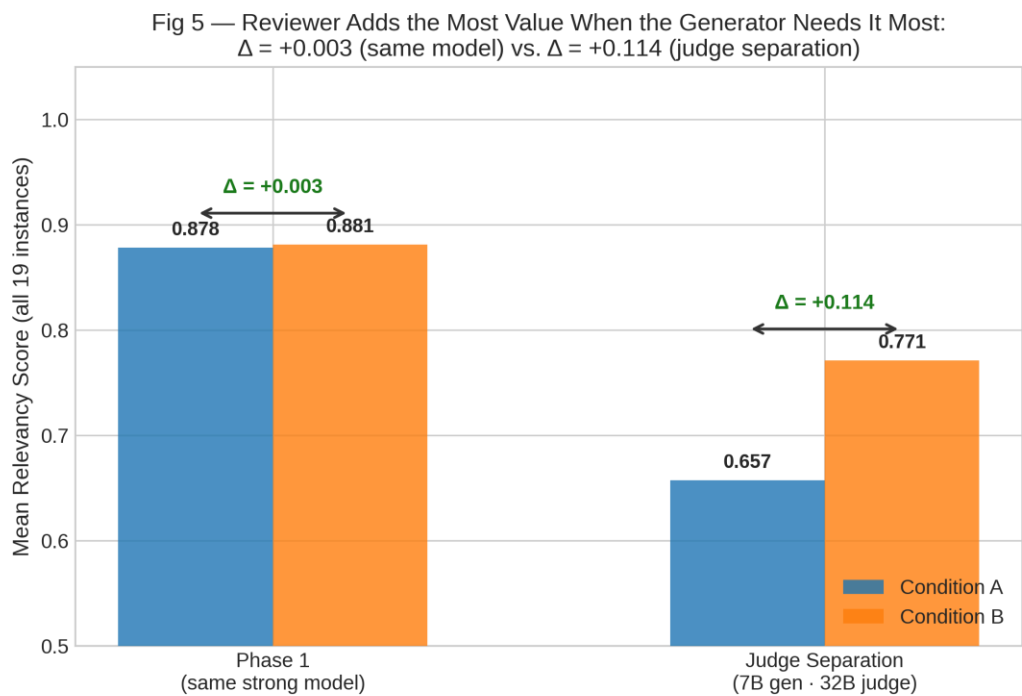


Figure 7. Judge separation comparison. Left: same strong model, $\Delta = +0.003$. Right: weak 7B generator vs strong 32B critic, $\Delta = +0.114$.

6.5 Summary of All Sweeps

Sweep	Condition	τ	Mean	σ	Min
Phase 1	A	0.70	0.878	0.145	0.30
Phase 1	B	0.70	0.881	0.069	0.72
Phase 1	C	0.70	0.887	0.126	0.40
Thresh B	B	0.85	0.926	0.044	0.85
Thresh C	C	0.85	0.721	0.353	0.00
Judge sep	A	—	0.657	0.270	0.20
Judge sep	B	0.85	0.771	0.218	0.20

Table 6. Summary of all five sweeps across 19 SWE-bench Lite instances.

7. Discussion

7.1 The Refine Loop as a Score Floor

The best discovery is that B conditions refine loop strictly realize the minimal score level. This is not a statistical trend but is rather a logical conclusion from the termination criterion of the algorithm. The loop is exited only if confidence $\geq$ threshold (accepted) or number of iterations has been reached. At threshold 0.85, all 19 instances were accepted before the cap was reached. The minimal score level guarantee thus held once in the present data.

The practical side of this is a little bit of imbalance: the Reviewer prevents the users from producing extremely low scores without improving the majority of the instances. In a production system with many users, the most important thing is to eliminate the tail (scores below 0.3) rather than shifting the median. That is precisely what the Reviewer does - the minimum A-condition of 0.30 becomes a minimum B-condition of 0.85.

7.2 Threshold Calibration Is Architecture-Dependent

The data indicate that the best threshold differs for Conditions B and C. In case of Condition B, thresholds getting higher generally mean improvement of scores up to the highest tested limit (0.85). For Condition C, the situation becomes the opposite after about 0.80: higher thresholds make the agent running the action worse by unnecessarily making it iterate longer on already long context traces.

Therefore, one single universal threshold is evidently not the right solution for systems with different architectures. A changing threshold considering the present context length or number of agent steps would enable Condition C to

get the Reviewer's error correction benefit on truly failing instances while not forcing iterations on instances that are already on the way to convergence.

7.3 Reviewer Contribution and Generator Quality

The judge separation experiment provides the answer to a question that Phase 1 data was incapable of answering. Does the Reviewer contribute value independently of generator quality, or is the Phase 1 gain ($\Delta = +0.048$) simply what any reasonably capable model does without the Reviewer? The answer is very evident from the 7b-generator outcome: the Reviewer adds $\Delta = +0.114$ when the generator is really having a hard time, while $\Delta = +0.003$ when the generator is already powerful.

This obviously affects deployment: teams running lightweight models on hardware limited by inference constraints will get much more benefit from the Reviewer than teams running cutting-edge models. The Reviewer is not unnecessary for strong models – it still makes a difference - but its additional value there is small.

7.4 Bimodality in Condition C

The C@0.85 bimodal distribution is the most structurally compelling finding. It means the OpenHands agent is behaving on these 19 instances more like a salt-and-pepper distribution rather than a single smooth curve: either the agent solves the problem and thus yields a high-quality trace (14 instances) or it finds itself in a dead end without being able to produce a good trace even if the Reviewer forces many refine iterations (5 instances).

The failing instances are not randomly picked - rather, they are instances of more complex multi-file dependency chains where the agent gets trapped in loop paths understanding, collecting context without moving forward.

7.5 Limitations

The evaluation is conducted on 19 instances. While directionally consistent across sweeps, 19 instances is insufficient for statistical significance claims about the full SWE-bench Lite distribution. Free-tier rate limiting introduced non-deterministic latency variance in Phase 1 runs, and three Condition C runs terminated mid-trace with transient network errors and were excluded. Absolute relevancy scores are sensitive to the judge model: switching from qwen3-32b to a different critic shifts scores, making cross-configuration comparisons unreliable without re-anchoring. The choice of the threshold at 0.85 was not empirically calibrated but chosen based on qualitative inspection of the Phase 1 distributions and should be validated on held-out instances.

8. Conclusions

This paper mainly details the conceptualization, implementation and experimental quantification over multiple sweeps of a minimal hallucination detection tool, which was basically a Reviewer Agent placed within the multi-agent architecture of the LangGraph system.

The principal result after five sweeps on 19 SWE-bench Lite units demonstrates that the score-motivated refine loop actually sets a strong score floor identical to the acceptance threshold. With a 0.85 threshold, average relevancy jumps from 0.878 to 0.926, and standard deviation decreases significantly from 0.145 to 0.044. The minimum single-instance value also goes up from 0.30 to 0.85.

The Hallucination Bridge takes the same reasoning and applies it to the very step-level action traces of an OpenHands automatic coding agent, thus supplying verifiable, forensically sound evidence for every action without the need for fine-tuning or direct logit access. The judge separation test reveals that the effect of the Reviewer gets bigger as the quality of the generator gets weaker: $\Delta = +0.003$ for an excellent generator versus $\Delta = +0.114$ for a bad one. The Condition C failure in bimodal manner at 0.85 threshold reveals that context explosion is the major failure mode for action-agent review, suggesting that adaptive threshold may be the very next engineering step.

9. Summary

Three concrete contributions were made this semester. First, a Reviewer Agent applying a five-dimension rubric (Correctness, Edge Cases, Security, Code Quality, Relevance) was implemented, producing schema-validated scores that gate a bounded refine loop. Second, the Hallucination Bridge extended the Reviewer's audit scope to the per-step action traces of an autonomous OpenHands coding agent. Third, five controlled experimental sweeps across 19 SWE-bench Lite instances provided empirical evidence for all conditions — the first multi-instance evaluation of the system.

The remaining work is a full 300-instance SWE-bench Lite evaluation for statistical significance, adaptive threshold calibration based on context length, and the migration to a paid OpenRouter account with per-account rate limits sufficient for sustained agentic workloads. The judge separation finding also motivates investigation of critic-generator pairings as a design variable: the data suggest that pairing any locally-feasible generator with a strong critic is a cost-effective strategy for improving output quality without increasing generator size.

10. Bibliography

- [1] Vaswani A. et al. Attention Is All You Need. arXiv:1706.03762, 2017.
- [2] Brown T.B. et al. Language Models are Few-Shot Learners. arXiv:2005.14165, 2020.
- [3] Ji Z. et al. Survey of Hallucination in Natural Language Generation. ACM Comput. Surv., 2023.
- [4] Huang L. et al. A Survey on Hallucination in LLMs. arXiv:2311.05232, 2023.
- [5] Maynez J. et al. On Faithfulness and Factuality in Abstractive Summarization. ACL, 2020.
- [6] Lin S. et al. TruthfulQA: Measuring How Models Mimic Human Falsehoods. ACL, 2022.
- [7] Manakul P. et al. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection. EMNLP, 2023.
- [8] Wu Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
- [9] Hong S. et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR, 2024.
- [10] Qian C. et al. ChatDev: Communicative Agents for Software Development. ACL, 2024.
- [11] Yao S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.

- [12] Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
- [13] Madaan A. et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023.
- [14] Jimenez C.E. et al. SWE-bench: Can LLMs Resolve Real-World GitHub Issues? ICLR, 2024.
- [15] Wang X. et al. OpenHands: An Open Platform for AI Software Developers. arXiv:2407.16741, 2024.
- [16] Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023.
- [17] Chen M. et al. Evaluating Large Language Models Trained on Code (Codex). arXiv:2107.03374, 2021.
- [18] Welleck S. et al. Generating Sequences by Learning to Self-Correct. ICLR, 2023.
- [19] Pan L. et al. Automatically Correcting Large Language Models. TACL, 2024.
- [20] Wang Y. et al. CodeT5+: Open Code Large Language Models. EMNLP, 2023.
- [21] Park J.S. et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST, 2023.
- [22] Liu A. et al. AgentBench: Evaluating LLMs as Agents. ICLR, 2024.

- [23] Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP. NeurIPS, 2020.
- [24] Schick T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.
- [25] Bairi R. et al. CodePlan: Repository-Level Coding using LLMs and Planning. FSE, 2024.
- [26] Ouyang L. et al. Training LMs to Follow Instructions with Human Feedback. NeurIPS, 2022.
- [27] Yang A. et al. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
- [28] Zhao W.X. et al. A Survey of Large Language Models. arXiv:2303.18223, 2023.